Advanced Python programming using AI tools

Lecture 10: Advanced OOP,
Comprehensions & Lambda function

Lecturer: Dr. Havana Rika

Today

1. Advanced OOP
2. Comprehensions
3. Lambda function

The Object Class

In Python, the object class is the **root of all class hierarchies**.

Every class inherits from this fundamental base class.

Key Characteristics of the object Class
Universal Base Class: The object class is at the top of the inheritance tree. This means every single data entity in Python is an object and possesses the basic properties and methods defined in the object class.

Minimal Functionality: It is a minimal, empty class that provides core, default behaviors for all objects, such as default implementations for special methods.

Default Special Methods: The object class provides default implementations for essential "dunder" (double underscore) methods, which can be overridden in subclasses:__init__, __str__, __eq__ , etc.

```python
class Person:  4 usages
    def __init__(self, name):
        self.name = name

    def info(self):
        return f"name={self.name}"

class Employee:  4 usages
    def __init__(self, salary):
        self.salary = salary

    def info(self):
        return f"salary={self.salary}"

class Worker(Person, Employee):  5 usages
    def __init__(self, name, salary):
        Person.__init__(self, name)
        Employee.__init__(self, salary)

w = Worker( name: "Dana",  salary: 10000)
print(isinstance(w, Worker), isinstance(w, Person), isinstance(w, Employee))
print(issubclass(Worker, Person), issubclass(Worker, Employee))
print(Worker.__bases__)
print(Person.__bases__)
```

```
True True True
True True
(<class '__main__.Person'>, <class '__main__.Employee'>)
(<class 'object'>,)
```

__bases__

משתנה של **מחלקה** (לא של מופע), שמחזיר tuple של מחלקות־העל **הישירות**.

issubclass()

checks if a specified class is a subclass (directly or indirectly) of another class or a tuple of classes.

```python
class Person:  4 usages
    def __init__(self, name):
        self.name = name

    def info(self):  1 usage
        return f"name={self.name}"

class Employee:  4 usages
    def __init__(self, salary):
        self.salary = salary

    def info(self):
        return f"salary={self.salary}"

class Worker(Person, Employee):  5 usages
    def __init__(self, name, salary):
        Person.__init__(self, name)
        Employee.__init__(self, salary)

w = Worker( name: "Dana",  salary: 10000)
print(w.info())
```

```
name=Dana
```

```python
class Person:  4 usages
    def __init__(self, name):
        self.name = name

    def info(self):
        return f"name={self.name}"

class Employee:  4 usages
    def __init__(self, salary):
        self.salary = salary

    def info(self):  1 usage
        return f"salary={self.salary}"

class Worker(Employee, Person):  5 usages
    def __init__(self, name, salary):
        Person.__init__(self, name)
        Employee.__init__(self, salary)

w = Worker( name: "Dana",  salary: 10000)
print(w.info())
```

```
salary=10000
```

Abstract Methods

```python
from abc import ABC, abstractmethod

class Shape(ABC):  1 usage
    @abstractmethod
    def area(self):
        pass

class Square(Shape):  1 usage
    def __init__(self, side):
        self.side = side
    def area(self):  1 usage
        return self.side * self.side

s = Square(3)
print(s.area())  # 9
```

Abstract Methods

```python
from abc import ABC, abstractmethod

class Shape(ABC):  1 usage
    @abstractmethod
    def area(self):
        pass

shape = Shape()  # This will raise an error
```

```
Traceback (most recent call last):
  File "C:\Program Files\JetBrains\PyCharm 2024.1.1\plugins\python-ce\helpers
    coro = func()
           ^^^^^^
  File "<input>", line 17, in <module>
TypeError: Can't instantiate abstract class Shape with abstract method area
```

Abstract Class

```
46  from abc import ABC, abstractmethod
47
48  # An abstract class with no abstract methods
49  class BaseClass(ABC):  1 usage
50      def common_method(self):  1 usage
51          print("This is a common, concrete method.")
52
53  # This class can be instantiated because there are no @abstractmethods
54  instance = BaseClass()
55  instance.common_method()
```

```
This is a common, concrete method.
```

The Magic Methods

```python
class Money:  4 usages
    def __init__(self, amount):
        self.amount = amount

    def __add__(self, other):
        return Money(self.amount + other.amount)

    def __sub__(self, other):
        return Money(self.amount - other.amount)

    def __eq__(self, other):
        return self.amount == other.amount

    def __lt__(self, other):
        return self.amount < other.amount

    def __str__(self):
        return f"{self.amount}₪"

m1 = Money(100)
m2 = Money(40)

print(m1 + m2)      # 140₪
print(m1 - m2)      # 60₪
print(m1 == m2)     # False
print(m2 < m1)      # True
```

https://docs.python.org/3/library/operator.html

Operation	Syntax	Function
Addition	`a + b`	`add(a, b)`
Concatenation	`seq1 + seq2`	`concat(seq1, seq2)`
Containment Test	`obj in seq`	`contains(seq, obj)`
Division	`a / b`	`truediv(a, b)`
Division	`a // b`	`floordiv(a, b)`
Bitwise And	`a & b`	`and_(a, b)`
Bitwise Exclusive Or	`a ^ b`	`xor(a, b)`
Bitwise Inversion	`~ a`	`invert(a)`
Bitwise Or	`a | b`	`or_(a, b)`
Exponentiation	`a ** b`	`pow(a, b)`
Identity	`a is b`	`is_(a, b)`
Identity	`a is not b`	`is_not(a, b)`
Identity	`a is None`	`is_none(a)`
Identity	`a is not None`	`is_not_none(a)`
Indexed Assignment	`obj[k] = v`	`setitem(obj, k, v)`
Indexed Deletion	`del obj[k]`	`delitem(obj, k)`
Indexing	`obj[k]`	`getitem(obj, k)`

https://docs.python.org/3/library/operator.html

Operation	Syntax	Function
String Formatting	`s % obj`	`mod(s, obj)`
Subtraction	`a - b`	`sub(a, b)`
Truth Test	`obj`	`truth(obj)`
Ordering	`a < b`	`lt(a, b)`
Ordering	`a <= b`	`le(a, b)`
Equality	`a == b`	`eq(a, b)`
Difference	`a != b`	`ne(a, b)`
Ordering	`a >= b`	`ge(a, b)`
Ordering	`a > b`	`gt(a, b)`

https://docs.python.org/3/library/operator.html

Operation	Syntax	Function
Left Shift	`a << b`	`lshift(a, b)`
Modulo	`a % b`	`mod(a, b)`
Multiplication	`a * b`	`mul(a, b)`
Matrix Multiplication	`a @ b`	`matmul(a, b)`
Negation (Arithmetic)	`- a`	`neg(a)`
Negation (Logical)	`not a`	`not_(a)`
Positive	`+ a`	`pos(a)`
Right Shift	`a >> b`	`rshift(a, b)`
Slice Assignment	`seq[i:j] = values`	`setitem(seq, slice(i, j), values)`
Slice Deletion	`del seq[i:j]`	`delitem(seq, slice(i, j))`
Slicing	`seq[i:j]`	`getitem(seq, slice(i, j))`

https://docs.python.org/3/library/operator.html

The Item Methods

```
class MyStore:  1 usage
    def __init__(self):
        self.data = {}

    def __getitem__(self, key):
        return self.data[key]

    def __setitem__(self, key, value):
        self.data[key] = value

    def __delitem__(self, key):
        del self.data[key]

store = MyStore()
store["a"] = 10
print(store["a"])   # 10
del store["a"]
```

The iter & next Methods

```python
my_list = [10, 20, 30]
my_iter = iter(my_list)
print(type(my_iter))


print(next(my_iter)) # Output: 10
print(next(my_iter)) # Output: 20
print(next(my_iter)) # Output: 30
print(next(my_iter)) # raise a StopIteration exception
```

```
<class 'list_iterator'>
10
20
30
Traceback (most recent call last):
  File "C:\Users\Havana\PycharmProjects\PythonCourse\program.py", line 9, in <module>
    print(next(my_iter)) # Output: 30
          ^^^^^^^^^^^^^
StopIteration

Process finished with exit code 1
```

The iter & next Methods

```
class Counter:  1 usage
    def __init__(self, n):
        self.n = n
        self.i = 0

    def __iter__(self):
        return self

    def __next__(self):
        if self.i >= self.n:
            raise StopIteration
        val = self.i
        self.i += 1
        return val

for x in Counter(3):
    print(x)  # 0 1 2
```

- `iter(obj)` returns an iterator. This is what `for` loops use.
- If an object defines `__iter__`, `iter(obj)` will call it.
- An iterator implements `__next__` to produce the next value, and ends by raising `StopIteration`.

The iter & next Methods

```
08  class Countdown:  1 usage
09      def __init__(self, start):
10          self.current = start
11
12      def __iter__(self):
13          return self
14
15      def __next__(self):
16          if self.current < 0:
17              raise StopIteration
18          val = self.current
19          self.current -= 1
20          return val
21
22  for x in Countdown(5):
23      print(x)
```

```
5
4
3
2
1
0
```

List Comprehension

```python
squares = [n*n for n in range(1, 6)]
print(squares)

squares = []
for n in range(1, 6):
    squares.append(n * n)
```

```
[1, 4, 9, 16, 25]
```

Dictionary Comprehension

{ key_expression : value_expression for element in iterable if condition }

```
squares = {n: n*n for n in range(1, 6)}
print(squares)

squares = {}
for n in range(1, 6):
    squares[n] = n * n
```

```
d = {"A": "Hello", "B": "WORLD"}
lowered = {k: v.lower() for k, v in d.items()}
print(lowered)

lowered = {}
for k, v in d.items():
    lowered[k] = v.lower()
```

```
{1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
{'A': 'hello', 'B': 'world'}
```

Dictionary Comprehension

{ key_expression : value_expression for element in iterable if condition }

```
17  d = {"a": 1, "b": 2, "c": 3, "d": 4}
18
19  only_even = {k: v for k, v in d.items() if v % 2 == 0}
20  print(only_even)
21
22  only_even = {}
23  for k, v in d.items():
24      if v % 2 == 0:
25          only_even[k] = v
```

```
{'b': 2, 'd': 4}
```

Set Comprehension

```
unique_lengths = {len(word) for word in ["Stop", "And", "Smell", "The", "Roses"]}
print(unique_lengths)
```

```
{3, 4, 5}
```

Lambda Function

A **lambda function** in Python is an **anonymous (nameless) small function** you write in a single expression.

It is most useful when you need a short function “on the spot”, **typically as an argument to another function**.

lambda parameters: expression

- No def name.
- The body must be one expression (not multiple statements).
- The expression’s value is automatically returned.

Lambda Function Examples

```
5   add = lambda a, b: a + b
6   print(add( a: 2,  b: 3))  # 5
7
8   x=add
9   print(x( a: 7,  b: 8))  # 15
0   print(add( a: 7,  b: 8))
1
2   def add(a, b):
3       return a + b
```

```
7   multiply = lambda a, b: a * b
8   print(multiply( a: 4,  b: 5))  # 20
9
10  def multiply(a, b):
11      return a * b
```

Why Use Lambda?

Use lambda when:

1. You need a **short** function for a one-time use.
2. A function requires a **callable** input (like key= in sorting).
3. You want to **avoid** creating a **separate named def** for something trivial.

Avoid lambda when:

1. The **logic** is more than very simple (readability drops quickly).
2. You need **multiple lines**, validations, logging, etc. (use def).

The Sorted Function

Syntax

```
sorted(iterable, key=key, reverse=reverse)
```

Parameter Values

Parameter	Description
iterable	Required. The sequence to sort, list, dictionary, tuple etc.
key	Optional. A Function to execute to decide the order. Default is None
reverse	Optional. A Boolean. False will sort ascending, True will sort descending. Default is False

Examples

```
24      pairs = [("b", 2), ("a", 10), ("c", 1)]
25      pairs_sorted = sorted(pairs, key=lambda x: x[1])
26      print(pairs_sorted)  # [('c', 1), ('b', 2), ('a', 10)]
27
```

```
[('c', 1), ('b', 2), ('a', 10)]
```

```
193     pairs = [("b", 2), ("a", 10), ("c", 1)]
194     def func(x):  1 usage
195         return x[1]
196     pairs_sorted = sorted(pairs, key=func)
197     print(pairs_sorted)
```

Examples

```
28  students = [{"name": "Dana", "grade": 88}, {"name": "Noam", "grade": 95}]
29  students_sorted = sorted(students, key=lambda s: s["grade"], reverse=True)
30  print(students_sorted)  # sorted by grade descending
```

```
[{'name': 'Noam', 'grade': 95}, {'name': 'Dana', 'grade': 88}]
```

```
6   students = [{"name": "Dana", "grade": 88}, {"name": "Noam", "grade": 95}]
7   def func(s):  1 usage
8       return s["grade"]
9   students_sorted = sorted(students, key=func, reverse=True)
0   print(students_sorted)  # sorted by grade descending
```